

Teleop Stack — Data-Interaction Map

Every input/output interaction between the **IK solver**, the **SpaceMouse group**, the **Quest**, and the **ZMQ interface** — ROS 2 (Jazzy) HIL teleop.

Scope: `ffw_collision_checker` (solver), `ffw_spacemouse` (mapper / base / quest bridge), `ffw_zmqinterface` (gateway). Top-level launch: `spacemouse_unified_teleop.launch.py`.

Generated 2026-09-04 · source-verified against current tree · all constants cross-checked from code

Read this first — one hard-won correction: the quest TRACK leash is **6 cm** (`kQuestMaxDist` = `0.06`), not 3 cm. APPROACH is 1.5 cm (`kQuestApproachMaxDist` = `0.015`). Any earlier note saying “3 cm TRACK” is stale.

1 · Subsystems and their roles

Subsystem	Package / executable	Role in the interaction graph
IK <code>ffw_ik_solver_teleop</code>	<code>ffw_collision_checker</code> · node <code>ffw_ik_solver_teleop</code>	MuJoCo IK engine, 100 Hz. Consumes both goal rails (<code>/spacemouse/<arm>/ee_target_pose</code> deltas and <code>/quest/<arm>/ee_target_pose</code> absolute poses), the quest per-arm state+trigger, raw <code>Joy</code> for buttons, <code>/joint_states</code> for real-state IK. Produces achieved EE poses, EE error, joint-trajectory commands, and the <code>/teleop/pose_changed</code> re-base signal.
SM Mapper (per arm)	<code>ffw_spacemouse</code> · <code>joy_node</code> + <code>joy_hand</code> , namespaces <code>left</code> / <code>right</code>	One <code>joy_hand</code> per arm reads the raw <code>Joy</code> of its SpaceMouse, applies soft locks + limit-profile clamps, and publishes incremental goal deltas. It is also the quest controller host : it runs the APPROACH/TRACK state machine on <code>/quest_state</code> telemetry and publishes the absolute quest goals on the second rail.
BASE <code>joy_base</code>	<code>ffw_spacemouse</code> · <code>joy_base</code>	Base/head teleop + ARM⇌BASE mode switching. Emits <code>/cmd_vel</code> , head trajectory, <code>/teleop_mode</code> , <code>/teleop_resync</code> , per-arm <code>precision_mode</code> Bool. Joint-state failsafe → full shutdown in hardware mode.
LOG <code>logitech_base_teleop.py</code> (optional)	<code>ffw_spacemouse</code> · script node + <code>joy_node_logitech</code>	Logitech Extreme 3D Pro (<code>046d:c215</code>) as an alternative base-control source. Re-publishes base <code>/cmd_vel</code> , <code>/Logitech/base_active</code> , and elevation joystick onto <code>/left/joy</code> .
Q <code>quest_to_ros2.py</code>	<code>ffw_spacemouse</code> · script, <code>ExecuteProcess</code>	Pure telemetry bridge (no control logic). Meta Quest → raw UDP CSV on :9500 → ROS: controller/head poses + <code>/quest_state</code> (38 floats). A second <code>joy_hand</code> -side stream only when <code>use_quest:=true</code> .
ZMQ <code>gateway_node</code>	<code>ffw_zmqinterface</code> · node <code>zmq_spacemouse_gateway</code>	The HIL link. Obs (EE+grippers+joints) PUB 6001, Priv (head-cam tf + per-tick EEDelta) PUB 6003, Control SUB 6002. Forwards ZMQ <code>ControlCmd</code> → <code>/quest/<arm>/ee_target_pose</code> + trigger (the same absolute rail the quest drives) and <code>OverrideCmd</code> → <code>/control_override</code> latch.
RW Peripheral / robot side	2× SpaceMouse (3Dconnexion 256f:c63a) · Meta Quest · real robot or Gazebo; optional baseline <code>robot_node.py</code> on the robot machine	Raw inputs to <code>joy_node(s)</code> and to the bridge; sink of trajectory commands; source of <code>/joint_states</code> . <code>robot_node.py</code> is the alternate ZMQ peer with synthetic Obs (no CONFLATE on Obs PUB, CONFLATE on Control SUB).

2 · Data-flow architecture



The three consumer rails the solver listens on are deliberately shaped: the SpaceMouse rail is **incremental**, the quest rail is **absolute**, and the gateway writes into the **quest** rail so an external controller and a physical quest can never both be live (override latch arbitrates). See §6.

3 · Subsystem I/O tables

IK 3.1 ffw_ik_solver_teleop — inputs

Topic	Type	Producer	Consumed as
-------	------	----------	-------------

/spacemouse/{left,right}/ee_target_pose	geometry_msgs/PoseStamped	joy_hand (SM path)	Incremental deltas: solver re-bases its internal goal by differencing consecutive goals (accum/leash invariant, \$6.1). Never treated as an absolute target.
/quest/{left,right}/ee_target_pose	geometry_msgs/PoseStamped	joy_hand (quest path) or gateway_node (ZMQ ControlCmd)	Absolute pose in frame <code>map</code> — replaces the internal goal directly, then leash-checked.
/quest/{left,right}/state	std_msgs/String	joy_hand	QuestState enum (SM_CONTROL / QUEST_APPROACH / QUEST_TRACK). Selects the leash: APPROACH 1.5 cm, TRACK 6 cm (\$6.4).
/quest/{left,right}/trigger	std_msgs/Float32	joy_hand or gateway_node	Normalized gripper command 0→1, honoured only while TRACK (\$6.5).
/({left,right})/joy	sensor_msgs/Joy	joy_node (namespace)	Buttons for non-TRACK gripper step open/close and misc control; not used for cartesian deltas.
/teleop_mode	std_msgs/String	joy_base (or logitech)	ARM/BASE switching context.
/joint_states	sensor_msgs/JointState	robot / Gazebo bridge	Real/current joint state for IK start point; feeds achieved-EE error.
/teleop_resync	std_msgs/Empty	joy_base	Re-sync IK after joint-state recovery.
/teleop_locks	std_msgs/String	supervisor	Comma list toggling joint soft locks (<code>soft_locked_joints_</code> , capture pending).
/dynamic_obstacle_pose	geometry_msgs/PoseStamped	obstacle tracker	Latest obstacle pose for collision checks during solve.
/leader/joystick_controller_left/joint_trajectory	trajectory_msgs/JointTrajectory	joy_base / logitech / solver head out	Head-joint feedback (<code>head_joint1/2</code> → <code>latest_head_pos_</code>), skipped while solving to home.

IK 3.2 ffw_ik_solver_teleop — outputs

Topic	Type	Consumer	Meaning
/ik_solver/achieved_ee_pose_{l,r}	geometry_msgs/PoseStamped	joy_hand (re-base, quest-engage age check), gateway_node (Obs EE block)	Solved EE pose after the controller step. Authoritative “where the arm actually is”.
/ik_solver/real_ee_error_{l,r}	std_msgs/Float32	supervisors / logging	Magnitude of current real-state→goal error.
/ik_solver/collision_debug	<i>ffw CollisionDebug</i>	diagnostics	Per-solve collision geometry debug.
/teleop/pose_changed	std_msgs/Empty	joy_hand	Fired when a pose load / home-reset homing completes → joy_hand re-bases <code>ee_goal_</code> onto the achieved pose.
/leader/joint_trajectory_command_broadcaster_{left,right}/joint_trajectory	trajectory_msgs/JointTrajectory	hardware controllers	Arm joint targets from IK.
/leader/joystick_controller_right/joint_trajectory	trajectory_msgs/JointTrajectory	hardware controllers	Lift trajectory.
/leader/joystick_controller_left/joint_trajectory	trajectory_msgs/JointTrajectory	head hardware + self-feedback sub	Head trajectory (also the solver’s own feedback topic, \$3.1).

SM 3.3 Mapper (joy_node + joy_hand, per arm) — inputs

Topic	Type	Producer	Consumed as
joy (<i>relative</i>) → /({left,right})/joy	sensor_msgs/Joy	joy_node in same namespace	Raw SpaceMouse axes/buttons (deadzone 0.0, autorepeat 100 Hz).
/quest_state	std_msgs/Float32MultiArray (38)	quest_to_ros2.py bridge	Quest telemetry, cached <code>quest_data_</code> with <code>quest_last_msg_time_</code> watchdog (timeout 0.5 s). Each arm instance picks its controller block (left idx 0 / right idx 16).
/teleop_mode	std_msgs/String	joy_base	Mode transitions reset quest SM to SM_CONTROL + disarm; BASE→ARM re-bases <code>ee_goal_</code> ← achieved pose.
/ik_solver/achieved_ee_pose_{l,r}	geometry_msgs/PoseStamped	solver	Re-base source; quest-engage gating (achieved age ≤ <code>quest_achieved_max_age_s</code> 0.5 s).
/teleop/pose_changed	std_msgs/Empty	solver	Re-base <code>ee_goal_</code> ← achieved pose after pose load / home reset.
/teleop_resync	std_msgs/Empty	joy_base	Joint-state recovery → re-base after achieved pose is stamped post-resync.
/ik_solver/ee_lock	std_msgs/String	supervisor	Soft-lock EE orientation capture on lock engage (orientation via extrinsic-XYZ RPY).
/teleop/limit_profile	std_msgs/String	supervisor	<code>set <arm> px_min .. yaw_max / clear <arm></code> — world-frame box, survives resyncs. Clamps <code>ee_target_delta</code> and the internal goals.
/spacemouse/<arm>/precision_mode	std_msgs/Bool	joy_base	Precision scaling of commanded deltas.
/control_override (<i>default topic</i>)	std_msgs/Bool	gateway_node (latch)	When TRUE: this joy_hand stops publishing /spacemouse/<arm>/ee_target_pose — the quest/ZMQ absolute rail wins in the solver. Release edge (TRUE→FALSE) re-bases.

SM 3.4 Mapper — outputs

Topic	Type	Consumer	Meaning / contract
/spacemouse/<arm>/ee_target_pose	geometry_msgs/PoseStamped	solver	Incremental goal rail (SM path). Suppressed entirely while <code>/control_override</code> is TRUE.
/spacemouse/<arm>/ee_target_delta	geometry_msgs/TwistStamped	gateway_node (Priv EEDelta), supervisors / demo loggers	Per-tick commanded delta after lock/limit-profile clamps: <code>linear = dx,dy,dz (m)</code> , <code>angular = drx,dry,drz (rad, extrinsic-XYZ)</code> . Explicit form of what the solver recovers by differencing — this topic is the authoritative “deltas out”.
/quest/<arm>/ee_target_pose	geometry_msgs/PoseStamped	solver (absolute rail)	Absolute map-frame goal computed from the quest controller pose while the quest state machine is active (APPROACH/TRACK).
/quest/<arm>/trigger	std_msgs/Float32	solver (TRACK-only gripper)	Live trigger analog, one value per quest tick. CTRL/APPROACH ignore it so the 4-way engage hold never slams the gripper.
/quest/<arm>/state	std_msgs/String	solver	One message per state change (SM_CONTROL / QUEST_APPROACH / QUEST_TRACK) so the solver picks the per-state leash.
/quest/<arm>/active	std_msgs/Bool	supervisors	Quest engaged flag for this arm.

/ffw_control/notification	std_msgs/String	HIL operator UI	Left mapper only (quest_publish_notifications false on right) — engage/disengage notifications.
---------------------------	-----------------	-----------------	--

Quest state machine (hosted in joy_hand): 4-way engage = both trigger analogs (offset 6) **and** both grip/thumb analogs (offset 7) ≥ 0.5 held $\geq$ `quest_hold_engage_s` (2.0 s) $\rightarrow$ QUEST_APPROACH (slow, `quest_w_slow` 0.15, converge within `quest_approach_pos_m` 0.05 m / `quest_approach_ang_rad` 0.10 rad) $\rightarrow$ QUEST_TRACK; release below `quest_release_min` 0.1 (with hysteresis 0.05) $\rightarrow$ SM_CONTROL. Velocity shaping: `quest_xy_scale` 3.0, `quest_pos_scale` 1.0, `quest_lead_t` 0.25 s, `quest_vel_alpha` 0.2.

BASE 3.5 joy_base — I/O

Direction	Topic	Type	Notes
IN	base_joy_topic (/right/joy) · aux_joy_topic (/left/joy)	sensor_msgs/Joy	Right SpaceMouse drives base; left is auxiliary (elevation etc.).
IN	/joint_states	sensor_msgs/JointState	QoS depth 20; joint-state failsafe gate inert in sim (joint_state_ready_ = !hardware_mode_).
IN	/logitech/base_active	std_msgs/Bool	Logitech active flag — base control source arbitration.
OUT	/cmd_vel	geometry_msgs/Twist	Base velocity commands.
OUT	head_topic (/leader/joystick_controller_left/joint_trajectory)	trajectory_msgs/JointTrajectory	Head pan/tilt (head_step 0.03125).
OUT	/teleop_mode	std_msgs/String	ARM/BASE mode; current_mode_ starts "ARM".
OUT	/teleop_resync	std_msgs/Empty	On joint-state recovery (consumed by mapper + solver).
OUT	/spacemouse/right/precision_mode · /spacemouse/left/precision_mode	std_msgs/Bool	Per-arm precision toggle consumed by each joy_hand.

LOG 3.6 logitech_base_teleop.py (optional) — I/O

Direction	Topic	Type	Notes
IN	/joy_logitech	sensor_msgs/Joy	joy_node_logitech remaps /joy $\rightarrow$ /joy_logitech , autorepeat 100 Hz.
OUT	cmd_vel_topic (/cmd_vel)	geometry_msgs/Twist	Base control override (max_linear_vel 0.6).
OUT	/logitech/base_active	std_msgs/Bool	Signal to joy_base.
OUT	head JointTrajectory · elevation_joy_topic (/left/joy)	JointTrajectory / Joy	Head + re-publishes elevation axis onto the left mapper's joy topic.
IN/OUT	mode_topic (/teleop_mode)	std_msgs/String	Shares the mode channel with joy_base.

Q 3.7 quest_to_ros2.py bridge — I/O

Direction / endpoint	Format	Consumer	Notes
IN — Meta Quest app	Raw UDP ASCII CSV, 0.0.0.0:9500 default (TCP localhost:8500 fallback)	—	One frame per line, CSV fields (§5.3). Pure telemetry.
OUT	/left_controller_pose · /right_controller_pose · /head_pose	diagnostics / TUI	Frame <code>map</code> . Unity $\rightarrow$ ROS remap: pos (pz, -px, py) , quat (qz, -qx, qy, -qw) .
OUT	/quest_state	joy_hand (both arms)	Float32MultiArray, 38 floats, layout §5.2. Published every tick (launch routes HUD to log, not console).

ZMQ 3.8 gateway_node — I/O

Direction	Topic / socket	Type	Notes
ROS IN	/ik_solver/achieved_ee_pose_{r,l}	geometry_msgs/PoseStamped	Source of Obs EE block (right = index 0, left = index 1).
ROS IN	/spacemouse/{0,1}/ee_target_delta	geometry_msgs/TwistStamped	Per-arm commanded deltas $\rightarrow$ Priv EEDelta, fresh-windowed (§6.7).
ROS IN	/joint_states	sensor_msgs/JointState	Relayed + reordered into JOINT_STATE_NAMES order (§5.1).
ROS IN	/head_camera_tf	tf message	Head-cam $\rightarrow$ control-frame transform for the Priv HeadCamTf block.
ZMQ IN	SUB tcp://*:6002 (single socket, SUBSCRIBE b"" , no CONFLATE)	ControlCmd / OverrideCmd	Dispatch by header msg_type ; stale-guarded by freshness (§6.6).
ZMQ OUT	PUB tcp://*:6001 (Obs)	proto.Obs	Every tick, once both achieved poses + first joint_state are present and finite (§6.8).
ZMQ OUT	PUB tcp://*:6003 (Priv)	proto.Priv	Fixed cadence once cam_tf known; EEDelta zero-filled per stale arm.
ROS OUT	/quest/right/ee_target_pose · /quest/left/ee_target_pose	geometry_msgs/PoseStamped	ControlCmd forward: frame <code>map</code> , header stamped now, quat via rpy_to_quat . Skipped when a byte-identical frame was already forwarded (_last_pub_data).
ROS OUT	/quest/right/trigger · /quest/left/trigger	std_msgs/Float32	Gripper = clamp01(gripper[i]) on the same trigger channel the quest drives. TRACK-only like the physical trigger.
ROS OUT	/control_override	std_msgs/Bool	Latched from OverrideCmd (≥ 0.5 = ON); mirrored each tick while fresh, self-releases on timeout (§6.6).

4 · Producer $\rightarrow$ consumer matrix (who talks to whom)

Producer	Consumer	Channel / topic	Type	Cadence	Carries
joy_node L / R	joy_hand L / R	/left/right/joy	Joy	100 Hz	Raw SpaceMouse axes + buttons (device deadzone 0.0).
joy_node R (base)	joy_base	/right/joy	Joy	100 Hz	Base drive axes, mode toggle.
joy_node L (aux)	joy_base · logitech elevation	/left/joy	Joy	100 Hz	Aux axis (elevation).
joy_hand (SM path)	solver	/spacemouse/<arm>/ee_target_pose	PoseStamped	mapper rate 100 Hz	Delta goal (accum/rebase on solver side). Stopped under control_override.
joy_hand	gateway · supervisors	/spacemouse/<arm>/ee_target_delta	TwistStamped	mapper rate	Post-clamp commanded delta → EEDelta on the ZMQ Priv rail.
joy_hand (quest path)	solver	/quest/<arm>/ee_target_pose	PoseStamped	quest state-machine rate	Absolute map-frame goal.
joy_hand	solver	/quest/<arm>/state	String	on change	QuestState → per-state leash.
joy_hand	solver	/quest/<arm>/trigger	Float32	each quest tick	Gripper (TRACK only).
joy_hand (L)	operator UI	/ffw_control/notification	String	events	Engage/disengage notices.
joy_base	joy_hand L/R, solver, logitech	/teleop_mode · /teleop_resync	String / Empty	events	ARM↔BASE mode; joint-state recovery.
joy_base	base hardware / Gazebo	/cmd_vel	Twist	100 Hz	Base velocities.
joy_base	head controller (+ solver feedback)	/leader/joystick_controller_left/joint_trajectory	JointTrajectory	on change	Head pan/tilt steps.
joy_base	joy_hand L/R	/spacemouse/{left,right}/precision_mode	Bool	events	Per-arm precision toggle.
logitech node	joy_base	/logitech/base_active	Bool	on change	Base control-source arbitration.
quest bridge	joy_hand L/R	/quest_state	Float32MultiArray (38)	~100 Hz	Both controllers + head telemetry (§5.2).
solver	joy_hand L/R	/ik_solver/achieved_ee_pose_{l,r}	PoseStamped	100 Hz	Re-base source; quest-engage age gate.
solver	joy_hand L/R	/teleop/pose_changed	Empty	events	Re-base after pose load / home reset.
solver	supervisor	/ik_solver/real_ee_error_{l,r} · collision_debug	Float32 / debug	100 Hz	Tracking error; collision diagnostics.
solver	arm/lift/head controllers	/leader/joint_trajectory_command_broadcaster_{left,right}/... · joystick_controller_{right,left}/joint_trajectory	JointTrajectory	100 Hz	IK joint targets (arms), lift, head.
robot / Gazebo	solver · joy_base · gateway	/joint_states	JointState	≥95 Hz hw gate	Current joint state (failsafe + IK start + Obs joint block).
cam tf bridge	gateway	/head_camera_tf	tf	resolved once	HeadCamTf for Priv.
gateway	external controller	ZMQ PUB 6001 · 6003	Obs / Priv	100 Hz	EE poses+grippers+joints; tf + EEDelta.
external controller	gateway	ZMQ SUB 6002	ControlCmd / OverrideCmd	client-rate	Absolute goals + grippers; override latch.
gateway	solver	/quest/<arm>/ee_target_pose + /quest/<arm>/trigger	PoseStamped / Float32	while cmd fresh	ZMQ ControlCmd injected onto the absolute rail.
gateway	joy_hand L/R (+ solver context)	/control_override	Bool	while latch fresh	Override gate: SM delta rail silenced, quest rail wins.

5 · Wire contracts

5.1 ZMQ framed structs — ffw_zmqinterface/protocol.py

One ZMQ message = one framed struct: fixed **12-byte header** + payload. ZMQ preserves message boundaries so no further framing is needed.

Header (every message):

[0:4]
type
int32
little-endian
-- MSG_* dispatch id

[4:12]
timestamp
float64
epoch seconds
-- sender clock; makes re-assertions unique for byte-dedup

RPY convention:
extrinsic XYZ → R = Rx(roll)·Ry(pitch)·Rz(yaw)
(matches joy_hand rpy_to_matrix)

MSG	id	Port	dir	Frame bytes	Payload layout
MSG_OBS	1	6001 (PUB)	robot → ctrl	724 B	89 doubles: EE block [0:14] = ee0(6) grip0 ee1(6) grip1 — ee0 = right , ee1 = left , grip = 0.0 open...1.0 close; joint block [14:89] = 25× pos, vel, effort in JOINT_STATE_NAMES order.
MSG_CONTROL_CMD	2	6002 (SUB)	ctrl → robot	124 B	14 doubles: ee0(6) grip0 ee1(6) grip1 — absolute EE goals + grippers (same 14-layout as Obs EE block).
MSG_PRIV	3	6003 (PUB)	robot → ctrl	236 B	28 doubles: HeadCamTf [0:16] row-major 4×4 (R 3×3 + translation col 3 + last row 0 0 0 1); EEDelta [16:28] = d0(6), d1(6) — per-tick map-frame commanded deltas (m / rad), all-zeros = no command.

MSG_OVERRIDE	4	6002 (SUB)	ctrl → robot	20 B	1 double: 0.0 OFF / 1.0 ON (≥ 0.5 = ON). Latched, self-releasing.
--------------	---	------------	--------------	------	---

JOINT_STATE_NAMES (fixed relay order, 25 joints; gripper indices bold)		
idx 0–7	arm_r_joint1..7 · gripper_r_joint1 (7)	right arm + gripper
idx 8–15	arm_l_joint1..7 · gripper_l_joint1 (15)	left arm + gripper
idx 16–24	lift_joint · head_joint1 · head_joint2 · left_wheel_steer · right_wheel_steer · rear_wheel_steer · left_wheel_drive · right_wheel_drive · rear_wheel_drive	lift, head, base wheels

Gateway reorders incoming `/joint_states` into this order (0.0-fill any missing joint), so the layout is stable regardless of broadcaster order. `effort` is measured Dynamixel current (A); wheel drives go through virtual_dxl and may read low-fidelity. Baseline `robot_node.py` mirrors the same ports: Obs PUB 6001 (no CONFLATE), ControlCmd SUB 6002 with `CONFLATE=1` (only newest control target matters).

5.2 /quest_state — 38-float layout

`quest_to_ros2.py` publishes `Float32MultiArray`; blocks at **LEFT_POSE_BEGIN 0, RIGHT_POSE_BEGIN 16, HEAD_POSE_BEGIN 32**. `joy_hand` selects its arm’s controller by offset (left 0, right 16, `QUEST_RIGHT_OFFSET`).

block offset	local idx	meaning
each controller block (16 wide)	0–5	pose: x, y, z, roll, pitch, yaw (map frame)
	6	trigger analog (0...1) — engage + gripper source
	7	grip / thumb analog (0...1) — 4-way engage co-requisite
	8	thumbstick_x
	9	thumbstick_y
	10	trigger_btn
	11	grip_btn
	12	a_x
	13	b_y
	14–15	thumbstick_btn · menu_btn
head block (32)	0–5	head pose x,y,z,roll,pitch,yaw; remaining floats unused

5.3 Quest UDP line-frames on :9500

Plain ASCII CSV, one frame per line. Head frame has 7 fields; each controller frame 13 fields:

```
Head pose:      px, py, pz, qx, qy, qz, qw
<Left|Right> controller: px, py, pz, qx, qy, qz, qw, trigger, grip, stick_x, stick_y, buttons, touches
Unity → ROS remap: position (pz, -px, py), quaternion (qz, -qx, qy, -qw)
```

6 · Gating & control semantics (the rules that make the data matter)

6.1 Delta-vs-absolute goal duality

`/spacemouse/<arm>/ee_target_pose` is **incremental** — the solver accumulates/re-bases by differencing consecutive goals, so a dropped goal only loses a step, never the reference. `/quest/<arm>/ee_target_pose` (whether from `joy_hand` or the gateway’s `ControlCmd` forward) is an **absolute** map-frame pose that replaces the internal goal directly. The mapper’s `/spacemouse/<arm>/ee_target_delta` publishes explicitly — after lock/limit-profile clamps — the same quantity the solver recovers internally, and it is that clamped delta that rides the ZMQ Priv rail as `EEDelta`. **Whichever topic delivered the last goal decides the leash.**

Invariant (accum/leash): `clip_target` must never re-base `accum_*` — that invariant is what makes the limit-profile box enforce correctly. Re-bases happen only at the events in §6.3.

6.2 control_override gate

When `/control_override` is **TRUE**, `joy_hand` **stops publishing** `/spacemouse/<arm>/ee_target_pose` entirely — the solver then sees only the absolute rail (`/quest/<arm>/ee_target_pose`), i.e. ZMQ/gateway or the physical quest. The override Bool is owned by the gateway: ZMQ clients send `MSG_OVERRIDE` instead of touching the topic directly.

6.3 Re-base events (ee_goal_ ← achieved pose)

- `BASE` → `ARM` switch (`/teleop_mode`)
- `/control_override` release edge (`TRUE` → `FALSE`)
- `/teleop/pose_changed` — pose load / home reset completed by the solver
- `/teleop_resync` — joint-state recovery (waits for an achieved pose stamped after the resync)
- Quest engage — but only if achieved pose age ≤ `quest_achieved_max_age_s` = 0.5 s

6.4 Solver leash constants (verified)

constant	value	meaning
kSpaceMouseMaxDist	0.01 m (1 cm)	max goal-step distance per tick on the SM delta rail
kQuestApproachMaxDist	0.015 m (1.5 cm)	per-tick leash while quest state = APPROACH
kQuestMaxDist	0.06 m (6 cm)	per-tick leash while quest state = TRACK (authoritative — not 3 cm)
kGoalMaxAngle	0.1 rad (≈ 5.7°)	per-tick goal rotation bound (all rails)
kMapperRebaseDist / kMapperRebaseAng	0.03 m / 0.1 rad	<code>joy_hand</code> ’s own re-base trigger when the goal runs away from achieved
kGripperLMin / kGripperLMax	0.0 / 1.2	left gripper physical range (rad) — model bg2
kGripperRMin / kGripperRMax	0.175 / 1.2	right gripper physical range (rad)

6.5 Gripper mapping (§11) — TRACK-gated

Read under `pose_mutex_`, applied only while the quest is in **TRACK** (in APPROACH the 4-way engage hold keeps trigger ≥ 0.5, so it is deliberately ignored; non-TRACK uses SpaceMouse button increments from `/left, right/joy`):

```
gripper_l_pos_ = clamp01(trigger_l)          * kGripperLMax      # 0.0 ... 1.2
gripper_r_pos_ = kGripperRMin + clamp01(trigger_r) * (kGripperRMax - kGripperRMin) # 0.175 ... 1.2
```

The gateway speaks the same normalized 0...1 trigger language (`clamp01`) and reports achieved openings back normalized against `_GRIP_OPEN` {0: 0.175, 1: 0.0} / `_GRIP_CLOSED` {0: 1.2, 1: 1.2} per arm index (0 = right, 1 = left).

6.6 Gateway freshness gates & latches

gate	window	behaviour
ControlCmd forward	cmd_timeout 0.5 s	Newest <code>_cmd</code> forwarded to <code>/quest/<arm>/ee_target_pose</code> + trigger only while <code>now - cmd_rcv_ts < 0.5 s</code> ; non-finite payload dropped; byte-identical re-forward suppressed via <code>_last_pub_data</code> (header ts keeps re-assertions unique).
EEDelta freshness	<code>_DELTA_FRESH_S</code> 0.25 s	Per-arm: a delta is relayed only if a new <code>ee_target_delta</code> arrived within 0.25 s of the last relay; otherwise that arm is zero-filled on the wire (all-zeros = “no command”). Frame still sent → fixed cadence once <code>cam_tf</code> is known.
Override latch	override_timeout 2.0 s	Mirror latched Bool each tick while fresh; when the <code>OverrideCmd</code> stream goes silent publish one final <code>False</code> and reset the latch — no stale <code>True</code> can resurrect.

6.7 Quest timeout & watchdog (joy_hand)

`quest_timeout_s` 0.5 s on `/quest_state` arrival (`quest_last_msg_time_`); a stale stream drops the quest back to `SM_CONTROL`. Engage hold: trigger (offset 6) and grip/thumb (offset 7) $\geq$ `quest_thumb/trigger_engage_min` (0.5) held $\geq$ 2.0 s; release $<$ 0.1 with 0.05 hysteresis.

6.8 Obs / Priv emission rules

Obs (6001): sent **only** when both achieved EE poses **and** the first `/joint_states` are present and all values finite — a zero-filled joint block would be worse than silence. Once resolved, one complete self-contained Obs per tick (joint block repeats newest `/joint_states` between broadcasts). **Priv (6003):** fixed cadence once `cam_tf` is known; EEDelta zero-filled per stale arm (§6.6). Both PUB rails have **no CONFLATE** — every tick is a complete observation for sync-and-concat consumers.

7 · Rates & timing summary

element	rate	notes
solver main loop	100 Hz	IK solve + controller step + publish
joy_node autorepeat / mapper	100 Hz / 100 Hz	<code>autorepeat_rate</code> 100.0, <code>publish_rate_hz</code> 100.0
joy_base / gateway	100 Hz	gateway hz 100.0
quest bridge HUD	~100 Hz	routed to launch log (not console)
hardware sync gate	$\geq$ 95 Hz <code>/joint_states</code> for 1 s	waits up to 30 s; base-joy failsafe → full Shutdown in <code>hardware_mode</code>

8 · Source map (where every claim lives)

- `ffw_collision_checker/src/ffw_ik_solver_teleop.cpp` — solver subs/pubs (pose, quest, joy, joints, locks, obstacle, head-feedback), leash constants, §11 gripper, `pose_changed` publisher.
- `ffw_spacemouse/src/joy_hand.cpp` — mapper topic construction, delta/absolute duality comments, quest SM + per-state leash, `control_override` gate, re-base events.
- `ffw_spacemouse/src/joy_base.cpp` — base/mode I/O, failsafe, precision Bools.
- `ffw_spacemouse/launch/spacemouse_unified_teleop.launch.py` · `spacemouse_mapper.launch.py` · `logitech_teleop.launch.py` — wiring, device detection, quest/gateway/logitech inclusion.
- `ffw_spacemouse/scripts/quest_to_ros2.py` · `logitech_base_teleop.py` — bridge + logitech node.
- `ffw_zmqinterface/ffw_zmqinterface/protocol.py` — wire structs, RPY, `JOINT_STATE_NAMES`.
- `ffw_zmqinterface/ffw_zmqinterface/gateway_node.py` — socket wiring, dispatch, forward/latch/tick rules, freshness windows.
- `ffw_zmqinterface/ffw_zmqinterface/robot_node.py` — baseline peer contract (CONFLATE asymmetry).